**CIÊNCIA DA COMPUTAÇÃO**

**DAVI DANDREA SIQUEIRA**
**JULIA ZAMONER**
**ALVARO DANKO**

# BIG O E COMPLEXIDADE DE ALGORITMOS:

um guia para entender quanto custa o seu código

## RESUMO

Este trabalho apresenta a notação Big O e a complexidade de algoritmos com linguagem acessível e comparações do cotidiano. Mostra por que um código que funciona bem com poucos dados pode ficar lento quando a entrada cresce, abordando complexidade de tempo e de espaço, regras de simplificação, as classes mais comuns, de O(1) a $O(2^n)$, com exemplos em Python, os casos melhor, pior e médio e as notações Ω (Ômega) e Θ (Theta). Também percorre a origem histórica do tema, sua relação com inteligência artificial, internet e segurança, descobertas recentes, limitações e curiosidades. Conclui-se que entender Big O permite escolher algoritmos e estruturas de dados mais adequados, resultando em sistemas mais rápidos e preparados para crescer.



**Palavras-chave:** algoritmos. complexidade computacional. notação Big O. estrutura de dados. desempenho.

## 1 INTRODUÇÃO

Quem já usou um aplicativo que funcionava bem no lançamento e depois começou a travar sabe que nem todo problema é erro de lógica. Muitas vezes o código está certo, só não foi pensado para crescer: com dez pedidos por dia qualquer delivery parece rápido, mas com dez mil a história muda. A análise de complexidade estima quanto trabalho um algoritmo, isto é, uma sequência de passos bem definidos para resolver um problema (CORMEN et al., 2012), exige conforme a entrada aumenta.

Para isso, usa-se a notação Big O, que funciona como o consumo de combustível na ficha técnica de um carro: não diz quanto tempo a viagem vai durar, mas mostra como o gasto cresce quando a estrada fica mais longa. O objetivo deste trabalho é explicar esse conceito com exemplos em Python e comparações do dia a dia, da origem do tema até suas aplicações atuais.

## 2 ORIGEM E EVOLUÇÃO DO TEMA

Curiosamente, a notação Big O nasceu antes do computador. O símbolo O apareceu em 1894, num livro de teoria dos números do matemático alemão Paul Bachmann, e a letra vem de *Ordnung*, “ordem” em alemão. Edmund Landau popularizou a notação em 1909, e por isso ela também é chamada de símbolo de Landau (KNUTH, 1976).

A computação adotou a ferramenta nos anos 1960. Em 1965, Juris Hartmanis e Richard Stearns fundaram a teoria da complexidade computacional, trabalho que lhes rendeu o Prêmio Turing (HARTMANIS; STEARNS, 1965), enquanto Alan Cobham e Jack Edmonds defendiam que um algoritmo eficiente é aquele de custo polinomial, como $n^2$, e não exponencial. Em 1976, Donald Knuth padronizou o uso de O, Ω e Θ na área (KNUTH, 1976).

Daí em diante, a pergunta passou de “quanto custa este algoritmo?” para “quanto custa, no mínimo, resolver este problema?”. Em 1971, Stephen Cook apresentou os problemas NP-completos, difíceis de resolver mas fáceis de conferir, e a questão “P é igual a NP?” virou o maior mistério da área, hoje um dos Problemas do Milênio, com prêmio de 1 milhão de dólares. O Quadro 1 resume esses marcos.

Quadro 1 – Marcos históricos da complexidade de algoritmos

| **Ano** | **Acontecimento** |
|---|---|
| 1894 | Paul Bachmann usa o símbolo O pela primeira vez |
| 1909 | Edmund Landau populariza a notação na matemática |
| 1936 | Alan Turing define formalmente o que é computar |
| 1965 | Hartmanis e Stearns fundam a teoria da complexidade computacional |
| 1971 | Stephen Cook apresenta os problemas NP-completos |
| 1976 | Donald Knuth padroniza O, Ω e Θ na computação |
| 1994 | Peter Shor cria um algoritmo quântico eficiente para fatorar números |
| 2000 | P vs NP entra na lista dos Problemas do Milênio |

Fonte: elaborado pelos autores com base em Knuth (1976) e Cormen et al. (2012).

## 3 COMPLEXIDADE E NOTAÇÃO BIG O

Complexidade é a quantidade de recursos que um algoritmo consome em relação ao tamanho da entrada, chamado de *n*: itens de uma lista, usuários cadastrados ou caracteres de um texto (ZIVIANI, 2011). Cronometrar o programa engana, porque o resultado muda de um computador para outro. Por isso, contam-se passos básicos, como comparações e acessos à memória.

### 3.1 Tempo e espaço

A complexidade de tempo mede como o número de operações cresce com *n*; a de espaço, quanta memória extra o algoritmo usa. Pense em lavar a louça: com dois pratos qualquer método serve, mas com a louça de uma festa o método faz diferença (tempo), e há quem use só a pia e quem espalhe pratos pela casa toda (espaço). Muitas vezes vale trocar memória por velocidade, como anotar uma conta no caderno para não refazê-la, técnica chamada *memoização*.

### 3.2 Definição

A notação Big O descreve um limite superior para o crescimento de uma função. Formalmente, f(n) é O(g(n)) quando existem constantes positivas $c$ e $n_0$ tais que (CORMEN et al., 2012):

$$0 \leq f(n) \leq c \cdot g(n), \text{ para todo } n \geq n_0 \qquad (1)$$

Ou seja, a partir de certo tamanho de entrada, f(n) nunca passa de uma versão multiplicada de g(n). É como numa maratona: na largada qualquer um pode sair na frente, mas depois de 40 km quem decide é o ritmo, e é esse ritmo que o Big O captura.

### 3.3 Regras para simplificar

Como só importa o comportamento para *n* grande, alguns detalhes podem ser descartados (SEDGEWICK; WAYNE, 2011). Constantes são ignoradas, então O(2n) vira O(n). Só o termo dominante fica, então $O(n^2 + n + 10)$ vira $O(n^2)$, assim como o preço de um chaveiro não muda o total de quem compra um carro. Trechos em sequência se somam e trechos aninhados se multiplicam: dois laços separados dão O(n), um laço dentro de outro dá $O(n^2)$. Por fim, entradas diferentes ganham letras diferentes: percorrer alunos (*a*) e, para cada um, suas disciplinas (*d*)

custa O(a × d).

```
def resumo_notas(notas):
    maior = notas[0]            # 1 passo
    for nota in notas:          # n passos
        if nota > maior:
            maior = nota
    for nota in notas:          # mais n passos
        print(nota)
    return maior                # 1 passo
```

No exemplo acima, a conta bruta é 2n + 2, que simplifica para O(n).

## 4 PRINCIPAIS CLASSES DE COMPLEXIDADE

As classes abaixo vão da mais eficiente para a mais custosa.

### 4.1 Constante, logarítmica e linear

No tempo constante, O(1), o custo não muda com a entrada, como pegar o controle remoto que você sabe estar na mesa de centro. Acessar uma lista pelo índice ou um dicionário do Python pela chave (em média) são exemplos.

No tempo logarítmico, O(log n), cada passo corta o problema pela metade, como adivinhar um número de 1 a 100 ouvindo só “maior” ou “menor”: chutando sempre no meio, bastam 7 tentativas, e para 1 milhão bastam cerca de 20. É o caso da busca binária, que exige lista ordenada (BHARGAVA, 2017):

```
def busca_binaria(lista_ordenada, alvo):
    inicio, fim = 0, len(lista_ordenada) - 1
    while inicio <= fim:
        meio = (inicio + fim) // 2
        if lista_ordenada[meio] == alvo:
            return meio
        if lista_ordenada[meio] < alvo:
            inicio = meio + 1
        else:
            fim = meio - 1
    return -1
```

No tempo linear, O(n), o custo cresce na mesma proporção da entrada, como procurar uma camiseta numa pilha bagunçada olhando peça por peça. É o caso de percorrer uma lista inteira com um `for`.

### 4.2 Linearítmica: O(n log n)

É a marca dos bons algoritmos de ordenação, como o Merge Sort, que divide a lista em metades, ordena as partes e depois as intercala. Lembra uma professora que divide 200 provas entre monitores e depois junta as pilhas já ordenadas. A função `sorted()` do Python já trabalha nessa classe.

### 4.3 Quadrática: O(n²)

Aparece quando cada item é comparado com todos os outros, como numa dinâmica em que todos da turma se cumprimentam: 10 pessoas dão 45 apertos de mão; 100 pessoas, 4.950.

```
def tem_repetido_lento(lista):          # O(n²)
    for i in range(len(lista)):
        for j in range(i + 1, len(lista)):
            if lista[i] == lista[j]:
                return True
    return False
```

Com um conjunto (`set`), que verifica presença em tempo constante na média, o mesmo problema cai para O(n), trocando memória por tempo:

```
def tem_repetido_rapido(lista):         # O(n)
    vistos = set()
    for item in lista:
        if item in vistos:
            return True
        vistos.add(item)
    return False
```

### 4.4 Exponencial e fatorial: O(2ⁿ) e O(n!)

No tempo exponencial, cada item a mais dobra o trabalho, como uma corrente em que cada pessoa repassa a mensagem para outras duas. O cálculo recursivo de Fibonacci é o exemplo clássico, e a memoização o reduz para O(n), pois cada valor passa a ser calculado uma vez só:

```
from functools import lru_cache

@lru_cache(maxsize=None)   # sem esta linha, o custo é O(2ⁿ)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Pior ainda é o tempo fatorial, O(n!), de quem testa todas as ordens possíveis, como todas as rotas entre várias cidades: com 20 cidades, são mais de 2 quintilhões.

### 4.5 Comparando o crescimento

A Tabela 1 mostra quantas operações, aproximadamente, cada classe exige conforme a entrada cresce.

Tabela 1 – Número aproximado de operações por classe de complexidade

| **Classe** | **n = 10** | **n = 100** | **n = 1.000** | **n = 1.000.000** |
|---|---|---|---|---|
| O(1) | 1 | 1 | 1 | 1 |
| O(log n) | 3 | 7 | 10 | 20 |
| O(n) | 10 | 100 | 1.000 | 1.000.000 |
| O(n log n) | 33 | 664 | 9.966 | 19.931.569 |
| $O(n^2)$ | 100 | 10.000 | 1.000.000 | $10^{12}$ |
| $O(2^n)$ | 1.024 | $1{,}27 \times 10^{30}$ | $1{,}07 \times 10^{301}$ | inviável |

Fonte: elaborada pelos autores (2026), com logaritmos na base 2.

Num computador que faz 1 bilhão de operações por segundo, um algoritmo $O(n^2)$ com 1 milhão de itens levaria cerca de 17 minutos; um $O(2^n)$ com apenas 100 itens, uns 40 trilhões de anos, quase 3 mil vezes a idade estimada do universo.

# 5 ANÁLISE NA PRÁTICA

## 5.1 Roteiro de análise

Na maioria dos casos, basta identificar o *n*, procurar os laços (um laço que percorre tudo é O(n), e laços aninhados se multiplicam) e ficar atento a trechos que cortam o problema pela metade, sinal de log n. O que mais engana são funções e operadores prontos, que podem esconder um laço inteiro:

```
def remover_inativos(usuarios, inativos):
    ativos = []
    for usuario in usuarios:
        if usuario not in inativos:   # inativos é uma lista
            ativos.append(usuario)
    return ativos
```

Parece O(n), mas o operador `in` percorre a lista `inativos` a cada usuário, resultando em O(u × i). Convertendo para `set`, o custo cai para O(u + i):

```
def remover_inativos(usuarios, inativos):
    inativos_set = set(inativos)
    return [u for u in usuarios if u not in inativos_set]
```

Complexidade ruim também é risco de segurança: quem consegue enviar uma entrada enorme para uma rotina $O(n^2)$ ou pior pode sobrecarregar o servidor, num ataque de negação de serviço. Por isso, além de bons algoritmos, convém limitar o tamanho das entradas aceitas.

## 5.2 Melhor, pior e caso médio

O desempenho também depende da entrada. Numa busca linear, achar o item na primeira posição é o melhor caso, O(1); não achá-lo é o pior caso, O(n); e, em média, são cerca de n/2 comparações, o que ainda é O(n). É como procurar as chaves de casa: às vezes estão no bolso, às vezes você revira tudo. No mercado, o pior caso costuma pesar mais, por funcionar como garantia.

Além do Big O, que é um limite superior (“no máximo”), existem a notação Ω (Ômega), limite inferior (“no mínimo”), e a Θ (Theta), usada quando os dois coincidem (CORMEN et al., 2012; KNUTH, 1997). Pense no GPS: “chega em no máximo 3 horas” é Big O, “leva pelo menos 1 hora” é Ômega e “cerca de 2 horas” é Theta. O Merge Sort, por exemplo, é Θ(n log n) em qualquer caso, enquanto a busca linear é O(n) e Ω(1).

## 6 BIG O NO MUNDO ATUAL

Na Inteligência Artificial, o mecanismo de atenção dos Transformers, base dos modelos de linguagem atuais, compara cada palavra com todas as outras, um custo $O(n^2)$ em relação ao tamanho do texto (VASWANI et al., 2017). É a lógica dos apertos de mão da seção 4.3, e explica por que textos longos ficam caros tão rápido. Na internet, aplicativos de mapas usam algoritmos de caminho mínimo como o de Dijkstra, bancos de dados encontram registros com índices em árvore, em O(log n), e sites guardam sessões em tabelas hash, com acesso médio O(1).

Na segurança, a criptografia RSA depende de não existir algoritmo clássico eficiente para fatorar números grandes. Como o algoritmo quântico de Shor faria isso em tempo polinomial, o mundo já migra para a criptografia pós-quântica. No sentido oposto, complexidade ruim causa falhas: em 2 de julho de 2019, uma expressão regular com retrocesso exponencial esgotou a CPU dos servidores da Cloudflare no mundo todo e deixou fora do ar, por 27 minutos, os sites que dependiam da empresa (GRAHAM-CUMMING, 2019).

A área também segue produzindo novidades. Em 2025, Ryan Williams, do MIT, provou que uma computação de tempo *t* cabe em memória da ordem de $\sqrt{(t \log t)}$, superando um resultado de 1975 (WILLIAMS, 2025). No mesmo ano, o grupo de Ran Duan, da Universidade Tsinghua, criou um algoritmo de caminho mínimo mais rápido que o de Dijkstra em grafos esparsos, $O(m \log^{2/3} n)$, quebrando a chamada “barreira da ordenação” e vencendo o prêmio de melhor artigo do STOC 2025 (DUAN et al., 2025). Em 2026, o mesmo grupo chegou a algo próximo de $O(m \sqrt{\log n})$ (DUAN et al., 2026).

## 7 LIMITAÇÕES E DESAFIOS

Ao descartar constantes, o Big O pode esconder custos importantes. O algoritmo de Duan et al. (2025), por exemplo, perdeu para o Dijkstra tradicional em testes com dezenas de milhões de vértices, por ter constantes bem maiores (CASTRO; CLEMENTINO; FREITAS, 2025). Há até algoritmos apelidados de “galácticos”, que só vencem com entradas grandes demais para existirem na prática.

A notação também ignora o hardware: percorrer um vetor, com dados lado a lado na memória, costuma ser mais rápido do que percorrer uma lista encadeada, embora ambos sejam O(n). E o pior caso pode enganar: o Quicksort é $O(n^2)$ no pior caso, mas está entre os mais rápidos na prática, e o Timsort do Python usa a ordenação por inserção, $O(n^2)$, em trechos curtos, porque ela vence em entradas pequenas.

Restam ainda desafios em aberto, como analisar sistemas paralelos e distribuídos e resolver P vs NP. Enquanto isso, problemas de logística e agendamento sem solução eficiente conhecida são atacados com heurísticas, que dão respostas boas, mas não necessariamente ótimas.

## 8 CURIOSIDADES E PRODUÇÕES SOBRE O TEMA

Donald Knuth envia um cheque de 2,56 dólares, o “dólar hexadecimal”, a quem encontra erros em seus livros, e muita gente prefere emoldurá-lo a descontá-lo. O Timsort foi criado em 2002 por Tim Peters para o Python e acabou adotado também pelo Java e pelo Android. E há algoritmos feitos só de piada, como o Bogosort, que embaralha a lista até ela ficar ordenada por acaso, com custo médio em torno de O(n × n!). O tema aparece ainda em filmes, séries e livros, reunidos no Quadro 2.

Quadro 2 – Produções que abordam complexidade de algoritmos

| **Produção** | **Tipo e ano** | **Relação com o tema** |
|---|---|---|
| O Jogo da Imitação | Filme, 2014 | Mostra Alan Turing tentando quebrar a Enigma, cujas combinações eram numerosas demais para testar uma a uma |
| Travelling Salesman | Filme, 2012 | Quatro matemáticos provam que P = NP e discutem as consequências com o governo |
| Os Simpsons, “Homer³” | Série, 1995 | A equação P = NP aparece escondida no cenário quando Homer entra na terceira dimensão |
| Elementary, “Solve for X” | Série, 2013 | Sherlock Holmes investiga assassinatos ligados à corrida pela solução de P vs NP |
| The Secret Rules of Modern Living: Algorithms | Documentário da BBC, 2015 | Explica como algoritmos de busca, ordenação e otimização estão presentes no cotidiano |
| Algoritmos para viver | Livro, 2016 | Aplica ideias de algoritmos a decisões do dia a dia, como organizar a casa ou escolher um apartamento |

Fonte: elaborado pelos autores (2026).

## 9 CONSIDERAÇÕES FINAIS

A notação Big O é uma forma de antecipar o futuro de um código: não informa quanto tempo um programa leva hoje, mas mostra o que acontece quando usuários e dados começam a crescer. Escolhas simples, como trocar uma lista por um conjunto ou guardar resultados já calculados, podem transformar um algoritmo quadrático ou exponencial em linear, e algoritmos que crescem sem controle podem gerar travamentos e até ataques.

Isso não significa otimizar tudo ao máximo. Para uma lista que nunca passa de vinte itens, código legível vale mais que solução esperta. A principal lição do Big O é reconhecer quando o tamanho da entrada importa e, nesses casos, escolher um caminho que continue funcionando quando a demanda aumentar.

## REFERÊNCIAS

BHARGAVA, Aditya Y. **Entendendo algoritmos**: um guia ilustrado para programadores e outros curiosos. São Paulo: Novatec, 2017.

CASTRO, Lucas; CLEMENTINO, Thailsson; FREITAS, Rosiane de. **Implementation and brief experimental analysis of the Duan et al. (2025) algorithm for single-source shortest paths**. arXiv, 2025. Disponível em: https://arxiv.org/abs/2511.03007. Acesso em: 29 out. 2026.

CORMEN, Thomas H. et al. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

DUAN, Ran et al. **Breaking the sorting barrier for directed single-source shortest paths**. arXiv, 2025. Disponível em: https://arxiv.org/abs/2504.17033. Acesso em: 29 out. 2026.

DUAN, Ran et al. A faster directed single-source shortest path algorithm. In: INTERNATIONAL COLLOQUIUM ON AUTOMATA, LANGUAGES, AND PROGRAMMING, 53., 2026. **Proceedings** [...]. Wadern: Schloss Dagstuhl, 2026. (LIPIcs, v. 374). Disponível em: https://arxiv.org/abs/2602.07868. Acesso em: 29 out. 2026.

GRAHAM-CUMMING, John. **Details of the Cloudflare outage on July 2, 2019**. Cloudflare Blog, 12 jul. 2019. Disponível em: https://blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-2019. Acesso em: 29 out. 2026.

HARTMANIS, Juris; STEARNS, Richard E. On the computational complexity of algorithms. **Transactions of the American Mathematical Society**, Providence, v. 117, p. 285-306, 1965.

KNUTH, Donald E. Big Omicron and big Omega and big Theta. **ACM SIGACT News**, New York, v. 8, n. 2, p. 18-24, 1976.

KNUTH, Donald E. **The art of computer programming**: fundamental algorithms. 3. ed. Reading: Addison-Wesley, 1997. v. 1.

SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Boston: Addison-Wesley, 2011.

VASWANI, Ashish et al. Attention is all you need. In: CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 31., 2017, Long Beach. **Proceedings** [...]. Red Hook: Curran Associates, 2017. p. 5998-6008.

WILLIAMS, R. Ryan. Simulating time with square-root space. In: ANNUAL ACM SYMPOSIUM ON THEORY OF COMPUTING, 57., 2025, Praga. **Proceedings** [...]. New York: ACM, 2025. p. 13-23. Disponível em:

https://doi.org/10.1145/3717823.3718225. Acesso em: 29 out. 2026.

ZIVIANI, Nivio. **Projeto de algoritmos**: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.